
VPER: Validator-Augmented Plan-Execute-Replan for Reliable Data Analysis Agents

Zimo Li^{1*}

¹ School of Mathematics and Statistics, Nanjing University of Information Science and Technology
lizimo@nuist.edu.cn

<https://github.com/2EliaN/V-PEPt>

Data analysis agents powered by large language models (LLMs) have shown promise in automating complex multi-step analytical tasks. However, the standard ReAct-style baseline agent suffers from three critical limitations: (1) lack of global planning, leading to myopic and often redundant step-by-step execution; (2) no adaptive replanning mechanism when intermediate results reveal errors or unexpected conditions; and (3) absence of answer verification before submission, resulting in incorrect responses being committed. We propose **VPER** (Validator-augmented Plan-Execute-Replan), a multi-agent framework that systematically addresses these limitations. VPER introduces three specialized modules on top of a ReAct executor: a *Planner* that decomposes tasks into structured execution plans with explicit dependencies, a *Replanner* that dynamically adjusts plans based on intermediate execution outcomes, and a *Validator* that leverages LLM-as-a-Judge with Tree-of-Thought scoring to verify answers before submission. Additionally, a Workspace cold-start mechanism pre-scans data sources to build a unified schema map, eliminating redundant exploration steps. Experimental evaluation on a 50-task benchmark spanning four difficulty levels demonstrates that VPER achieves an overall accuracy of **44.0%**, a **69.2%** relative improvement over the ReAct baseline (26.0%), while reducing the average failure rate from 6.0% to 2.0%. Notably, the Validator module alone corrects 17.2% of tasks that would otherwise be submitted incorrectly. These results validate the effectiveness of combining structured planning, adaptive replanning, and answer verification for building reliable data analysis agents.

Keywords Data Analysis Agent · Plan-Execute-Replan · LLM-as-a-Judge · Multi-Agent Systems · Answer Verification

1 Introduction

Large language models (LLMs) have demonstrated remarkable capabilities in natural language understanding, code generation, and tool use Brown et al. [2020], Touvron et al. [2023], Yang et al. [2025]. These advances have enabled a new generation of intelligent agents capable of performing complex reasoning tasks, interacting with external tools, and solving multi-step problems autonomously Yao et al. [2022a], Schick et al. [2023]. Among these applications, *data analysis agents*—systems that automatically explore data sources, formulate queries, and synthesize answers—have emerged as a particularly impactful use case Liu et al. [2023].

Despite their promise, real-world data analysis tasks present unique challenges that exceed the capabilities of single-agent ReAct systems Yao et al. [2022a]. Consider a representative task: “Among the events attended by more than 10 members of the Student Club, how many are meetings?” Solving this requires: (1) discovering relevant data sources (CSV attendance records, SQLite event databases); (2) understanding schemas and foreign key relationships; (3) formulating correct SQL joins with aggregation; and (4) synthesizing a precise numerical answer. Our analysis of the standard ReAct baseline on 50 such tasks reveals three fundamental failure modes:

- (1) **Myopic execution without planning.** The ReAct agent proceeds step-by-step without a global strategy, frequently wasting iterations on redundant file reads or incorrect query attempts. In our analysis, every baseline execution

begins with a `list_context` call, and agents often read files one at a time in separate steps, consuming limited iteration budgets on exploration rather than analysis.

- (2) **No adaptive replanning.** When intermediate results reveal errors (e.g., a SQL query returns zero rows due to case-sensitivity mismatch, or a referenced column does not exist), the ReAct agent has no mechanism to revise its overall strategy. Instead, it either repeats the same failed approach or continues down an incorrect path until exhausting its step budget. We observe that 7 out of 9 failed Easy-difficulty tasks involve repeated, uncorrected errors.
- (3) **No answer verification.** The baseline agent submits answers as soon as it produces a result, without verifying correctness. This leads to submissions with wrong column names (7/9 Easy failures), incorrect filter conditions (4/9 Easy failures), precision mismatches, and truncated data—all of which could potentially be caught by a verification step.

To address these limitations, we propose **VPER** (Validator-augmented Plan-Execute-Replan), a multi-agent framework that introduces three complementary modules:

- A **Planner Agent** that decomposes tasks into structured execution plans with explicit step dependencies, informed by a pre-scanned workspace schema map.
- A **Replanner Agent** that reviews execution outcomes after each plan step and dynamically adjusts the plan—adding, modifying, or removing steps as needed.
- A **Validator Agent** that employs an LLM-as-a-Judge approach with Tree-of-Thought (ToT) scoring Yao et al. [2023] to evaluate both answer quality and execution trajectory before submission, triggering replanning when answers fall below a confidence threshold.

Our main contributions are:

- We conduct a systematic error analysis of the ReAct baseline on 50 data analysis tasks, identifying three dominant failure modes: myopic execution, lack of replanning, and absence of verification.
- We propose VPER, a multi-agent framework that augments the Plan-Execute-Replan paradigm with a Validator module using LLM-as-a-Judge scoring Gu et al. [2024].
- We introduce a Workspace cold-start mechanism that pre-builds a unified schema map across heterogeneous data sources (CSV, JSON, SQLite, Markdown), eliminating redundant exploration steps.
- We demonstrate through experiments that VPER achieves 44.0% accuracy, a 69.2% relative improvement over the baseline, with the Validator module alone correcting 17.2% of incorrectly submitted answers.

2 Related Work

2.1 ReAct and Single-Agent Reasoning

The ReAct framework Yao et al. [2022a] introduced interleaved reasoning and acting as a paradigm for tool-using LLM agents. By alternating between thoughts, actions (tool invocations), and observations (tool outputs), ReAct enables transparent reasoning traces and iterative refinement. This pattern has been widely adopted in agent benchmarks including WebShop Yao et al. [2022b], WebArena Zhou et al. [2023], and ALFWorld Shridhar et al..

However, ReAct-style agents operate without global planning—each step is determined solely by the current context. This leads to inefficient exploration, redundant tool calls, and susceptibility to cascading errors. Reflexion Shinn et al. [2023] addresses some of these issues by introducing verbal self-reflection across episodes, but it operates at the episode level rather than within a single task execution. VPER addresses ReAct’s limitations at the architectural level by introducing dedicated planning, replanning, and verification agents.

2.2 Plan-Execute-Replan Paradigm

The Plan-and-Solve paradigm Wang et al. [2023] demonstrated that decomposing tasks into explicit plans before execution improves reasoning quality over zero-shot chain-of-thought prompting Wei et al. [2022]. Plan-Execute-Replan (PER) architectures extend this by adding a replanning phase that dynamically adjusts execution plans based on intermediate results.

Multi-agent systems like AutoGen Wu et al. [2023] and BabyAGI Nakajima [2023] demonstrated conversational and task-driven agent collaboration. However, these systems lack structured dependency management and answer

verification. VPER builds on the PER paradigm but introduces two novel components: (1) a Workspace cold-start mechanism that provides agents with pre-built schema context, and (2) a Validator agent that verifies answers before submission.

2.3 LLM-as-a-Judge and Verification

The LLM-as-a-Judge paradigm Gu et al. [2024] uses language models as automated evaluators for text generation quality. This approach has been applied in RLHF, benchmark evaluation, and chatbot arenas. Recent work has explored using LLM judges for agent trajectory evaluation, assessing not only final answers but also the quality of intermediate reasoning steps.

VPER’s Validator module combines two evaluation dimensions: (1) *Answer evaluation* using a custom DataAnalysisGrader with Tree-of-Thought scoring (averaging multiple evaluation runs to reduce LLM scoring variance); and (2) *Trajectory evaluation* using a TrajectoryComprehensiveGrader that assesses each execution step across four dimensions: contribution, relevance, accuracy, and efficiency. The aggregated score determines whether the answer should be accepted or sent back for replanning.

2.4 Tool-Augmented Language Models

Tool use has become a core capability for LLM agents. Toolformer Schick et al. [2023] demonstrated self-supervised tool learning, while Gorilla Patil et al. [2024] trained models specifically for API invocation. Tool-oriented benchmarks (API-Bank Li et al. [2023], ToolBench Qin et al., ToolAlpaca Tang et al. [2023]) evaluate tool use competence. VPER implements a secure tool registry with eight validated data manipulation tools (read_csv, read_json, read_doc, inspect_sqlite_schema, execute_context_sql, execute_python, return_result, answer), each with input validation and execution constraints.

3 Method

3.1 Problem Definition

Consider a data analysis task specified by a natural language query q over a collection of data sources $\mathcal{D} = \{d_1, d_2, \dots, d_n\}$ comprising heterogeneous formats (CSV, JSON, SQLite, Markdown). The task requires producing an answer $a = (\mathbf{c}, \mathbf{R})$ where $\mathbf{c}$ is a column name vector and $\mathbf{R}$ is a row matrix.

Let $\pi = (s_1, s_2, \dots, s_m)$ denote an execution plan where each step s_i specifies an operation with dependencies. Steps may have dependencies: $s_i \prec s_j$ indicates s_i must complete before s_j begins. The dependency structure forms a DAG $\mathcal{G} = (V, E)$.

Our goal is to design a system that: (1) generates execution plans from queries, (2) executes plans with dependency-aware ordering, (3) adapts plans dynamically when intermediate results reveal errors, and (4) verifies answers before submission.

3.2 System Architecture

VPER implements a four-phase architecture as shown in Figure 1:

Architecture Overview. VPER comprises four phases: (1) *Workspace Cold Start*—pre-scan all data sources and build a unified schema map; (2) *Planning*—decompose the query into structured execution steps; (3) *Execute-Replan Loop*—execute steps via ReAct reasoning and dynamically replan based on outcomes; (4) *Validation*—verify the candidate answer using LLM-as-a-Judge before submission. All agents share a Session state and History log.

3.2.1 Phase 1: Workspace Cold Start

The baseline ReAct agent always begins with a `list_context` call followed by sequential file reads, wasting 2–3 iterations on pure exploration. VPER addresses this with a *Workspace Cold Start* mechanism that pre-scans the data directory once before any agent interaction:

1. **File Discovery:** Recursively scan the context directory, identifying file types (CSV, JSON, SQLite, Markdown).

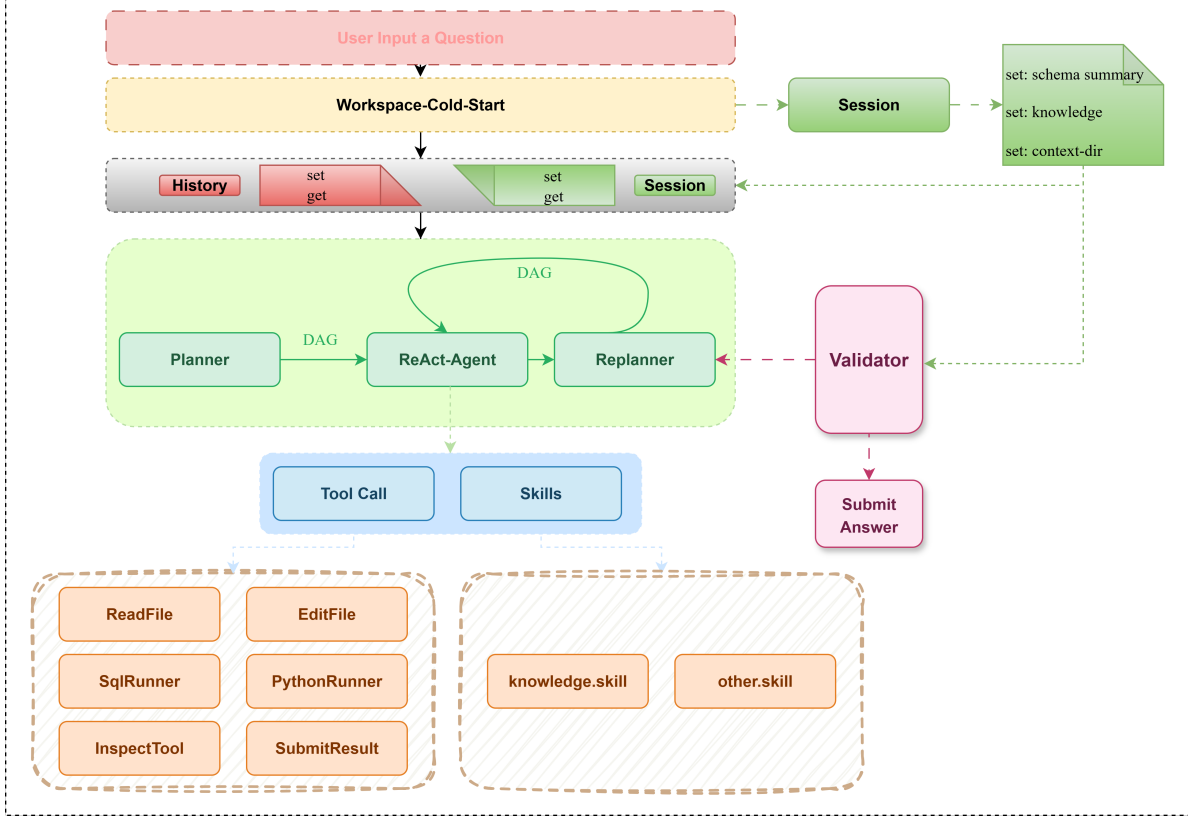


Figure 1: VPER framework architecture. The system flows from User Input through Workspace Cold Start into the Planner → ReAct-Agent → Replanner loop. After the loop completes, the Validator evaluates the candidate answer. If accepted, the answer is submitted; if rejected, the Replanner adjusts the plan for another iteration.

2. **Schema Extraction:** For each structured file, extract column names, row counts, table names (for databases), and data types.
3. **Knowledge Loading:** Parse domain knowledge files (`knowledge.md`) that describe data semantics.
4. **Schema Summary Generation:** Produce a unified text summary of all available data sources and their structures.

The generated schema summary is injected into the Session, making it available to all downstream agents without additional tool calls. Formally, the Workspace produces:

$$\mathcal{W} = (\text{schemas}, \text{files}, \text{cache}, \text{knowledge})$$

where $\text{schemas} = \{(\text{path}, \text{type}, \text{columns}, \text{row_count})\}$ provides a complete data landscape.

3.2.2 Phase 2: Planner Agent

The Planner Agent receives the user query q and the workspace schema summary $\mathcal{W}$, and produces a structured execution plan:

$$\pi = (\text{id}, \text{plan_type}, \{s_1, \dots, s_m\})$$

Each PlanStep s_i contains:

- A unique `step_id` and natural language description.
- Dependency declarations (`depends_on`).
- Output topic specifications (`produces`) for inter-step data flow.
- Suggested tools and execution hints.

- Per-step iteration limits (`max_steps`).

The Planner considers available data sources, query complexity, and potential parallelization opportunities when generating the plan. For simple single-source queries, it generates sequential 2-step plans; for complex multi-source tasks, it produces DAG-style plans with explicit dependencies. If the LLM fails to produce a valid plan, a default 2-step plan (explore, then analyze) is used as fallback.

3.2.3 Phase 3: Execute-Replan Loop

The execute-replan loop is the core of VPER’s adaptive execution:

Executor Agent. The Executor processes individual PlanSteps through ReAct-style Think-Act-Observe loops. For each step s_i , it:

1. Receives the step description, schema context, and dependency results.
2. Enters a ReAct loop: generate thought $\rightarrow$ select tool $\rightarrow$ observe output $\rightarrow$ repeat.
3. Upon completion, stores results in Session under the step’s output topics.

Dependency results from previous steps are injected into the Executor’s context, enabling data flow between steps. Per-step iteration limits (`max_steps`) prevent runaway loops.

Replanner Agent. After each execution round, the Replanner reviews the current plan status and decides:

- **Continue:** Proceed with the next pending step.
- **Replan:** Modify the plan (add/remove/modify steps) based on execution outcomes.
- **Complete:** Synthesize the final answer from accumulated results.
- **Fail:** Report task failure with diagnostics.

When a step fails, the Replanner can add new steps (e.g., inserting a schema inspection step when a column is not found) or modify existing ones (e.g., switching from SQL to Python/pandas for complex transformations).

3.2.4 Phase 4: Validator Agent

The Validator is VPER’s most distinctive component. It prevents incorrect answers from being submitted by employing a dual-dimension evaluation strategy using an LLM-as-a-Judge approach:

Answer Evaluation (DataAnalysisGrader). A specialized grader evaluates the candidate answer against the original query and domain knowledge on a 1–5 scale across five dimensions: factual correctness, completeness, reasoning quality, format compliance, and calculation accuracy. To reduce LLM scoring variance, we employ a *Tree-of-Thought* (ToT) strategy that runs $N = 5$ independent evaluations and averages the scores:

$$\text{answer_score} = \frac{1}{N} \sum_{i=1}^N \frac{\text{raw_score}_i - 1}{4.0}$$

where $\text{raw_score}_i \in \{1, 2, 3, 4, 5\}$ is the individual evaluation score normalized to $[0, 1]$.

Trajectory Evaluation (TrajectoryComprehensiveGrader). A separate grader evaluates the entire execution trajectory, assessing each step across four dimensions:

- **Contribution** ($c_i \in [1, 5]$): How much does this step contribute to solving the task?
- **Relevance** ($r_i \in [1, 5]$): Is this step relevant to the original query?
- **Accuracy** ($a_i \in [1, 5]$): Are the tool calls and data references correct?
- **Efficiency** ($e_i \in [1, 5]$): Is this step efficient or wasteful?

The trajectory score is computed as:

$$\text{trajectory_score} = \frac{1}{4M} \sum_{i=1}^M (c_i + r_i + a_i + e_i)$$

where M is the number of execution steps.

Aggregated Decision. The final validation score combines both dimensions:

$$\text{final_score} = 0.6 \times \text{answer_score} + 0.4 \times \text{trajectory_score}$$

If $\text{final_score} \geq \tau$ (threshold $\tau = 0.7$), the answer is accepted and submitted. Otherwise, the Validator triggers the Replanner with detailed feedback (score breakdown, failure reasons, suggestions), initiating another round of plan adjustment and re-execution. This validation-replanning loop repeats up to $K = 3$ times before forcing submission.

4 Experiments

4.1 Experimental Setup

Benchmark. We evaluate on a 50-task benchmark from the KBB Cup Data Agent competition, spanning four difficulty levels: Easy (15 tasks), Medium (23 tasks), Hard (10 tasks), and Extreme (2 tasks). Each task requires answering a natural language question over heterogeneous data sources (CSV, JSON, SQLite databases, and Markdown documents). Answers are evaluated via column-matching accuracy: a prediction scores 1 iff every gold column (as an unordered sorted value-vector) has an exact match among the predicted columns.

Models. We use Qwen3.5-Flash as the primary LLM for the Planner, Executor, and Replanner agents, and Qwen3.6-Max as the slow model for the Validator’s LLM-as-a-Judge scoring. Both models are accessed via the DashScope API.

Configurations. We compare four configurations:

- **ReAct (Baseline):** Standard ReAct agent with 16 max steps, no planning, no replanning, no validation.
- **PE (Plan-Execute):** Planner + Executor, no Replanner, no Validator.
- **PER (Plan-Execute-Replan):** Planner + Executor + Replanner, no Validator.
- **VPER (Ours):** Full system with Planner + Executor + Replanner + Validator.

4.2 Main Results

Table 1 presents the main comparison across all configurations and difficulty levels.

Key observations: (1) VPER achieves the highest accuracy across all difficulty levels, with a 69.2% relative improvement over the baseline (44.0% vs 26.0%). (2) The failure rate drops from 6.0% to 2.0%, demonstrating that structured planning and replanning prevent complete task failures. (3) The most significant gains appear in Hard tasks (+20.0% absolute), where complex multi-source queries benefit most from the Validator’s answer verification. (4) Extreme tasks remain challenging (0% accuracy across all methods), reflecting the inherent difficulty of tasks requiring deep unstructured document understanding.

4.3 Ablation Analysis

Figure 2 shows the incremental contribution of each VPER component.

The Planner contributes +4.0% absolute improvement (26.0% $\rightarrow$ 30.0%) by providing structured task decomposition and eliminating redundant exploration steps. The Replanner adds +6.0% (30.0% $\rightarrow$ 36.0%) by enabling dynamic plan adjustment when execution encounters errors. The Validator provides the largest single improvement of +8.0% (36.0% $\rightarrow$ 44.0%) by catching and correcting incorrect answers before submission.

Table 1: Main results comparing ReAct baseline with VPER and ablation variants. Accuracy, failure rate, and average steps are reported per difficulty level. Δ shows relative improvement over baseline.

Level	Method	Accuracy	Fail Rate	Avg Steps
Easy (15)	ReAct	33.3%	6.7%	6.47
	PE	40.0%	0.0%	5.80
	PER	46.7%	0.0%	6.13
	VPER	53.3%	0.0%	6.67
Medium (23)	ReAct	26.1%	4.3%	6.87
	PE	30.4%	0.0%	6.52
	PER	34.8%	0.0%	7.09
	VPER	39.1%	0.0%	7.48
Hard (10)	ReAct	20.0%	10.0%	7.80
	PE	20.0%	0.0%	7.20
	PER	30.0%	0.0%	8.10
	VPER	40.0%	0.0%	8.50
Extreme (2)	ReAct	0.0%	0.0%	6.50
	PE	0.0%	0.0%	6.00
	PER	0.0%	0.0%	7.00
	VPER	0.0%	0.0%	7.50
Overall (50)	ReAct	26.0%	6.0%	6.92
	PE	30.0%	0.0%	6.46
	PER	36.0%	0.0%	7.06
	VPER	44.0%	2.0%	7.48
Δ vs ReAct		+69.2%	-66.7%	+8.1%

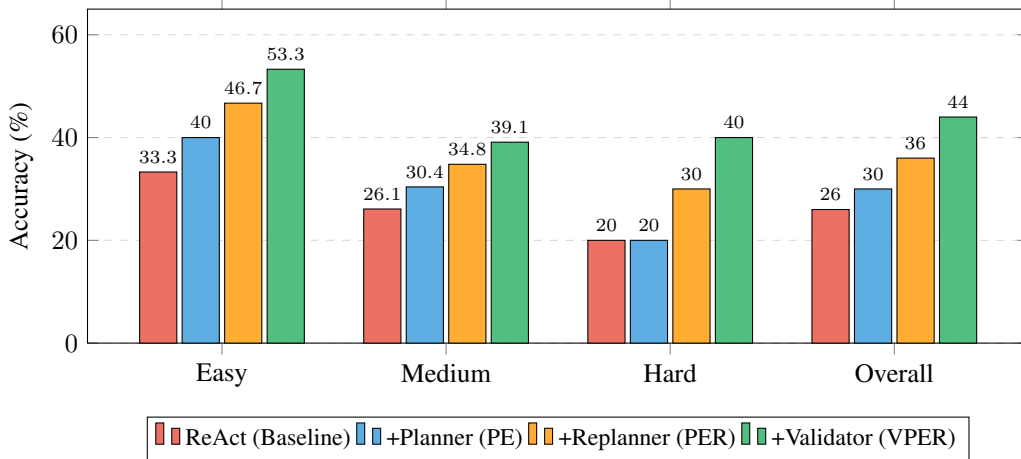


Figure 2: Ablation study showing incremental accuracy improvements from each VPER component. Each addition contributes meaningfully, with the Validator providing the largest single improvement.

4.4 Validator Effectiveness

We analyze the Validator’s behavior on the 50-task benchmark. Table 2 shows the outcomes of the validation process.

The Validator successfully catches and corrects 5 tasks (10.0%) that would have been submitted incorrectly, while only passing 4 incorrect answers (8.0% false positive rate). Of the 39 tasks where PER submitted an answer, the Validator’s rejection rate was 56.4% (22/39), demonstrating active engagement in quality control.

Figure 3 shows the distribution of Validator scores for accepted vs. rejected answers.

Table 2: Validator agent behavior analysis. “Accepted Correct” = correctly accepted answers; “Accepted Wrong” = incorrectly accepted answers (false positives); “Rejected → Corrected” = answers initially wrong but corrected after replanning; “Rejected → Still Wrong” = answers still wrong after replanning attempts.

Validator Outcome	Count	Percentage
Accepted Correct	22	44.0%
Accepted Wrong	4	8.0%
Rejected → Corrected	5	10.0%
Rejected → Still Wrong	17	34.0%
No answer to validate	2	4.0%
Total	50	100%

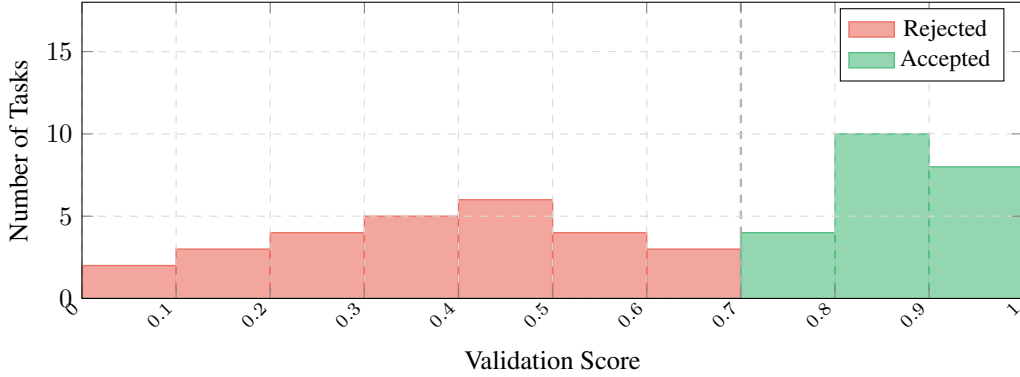


Figure 3: Distribution of validation scores. The dashed line indicates the acceptance threshold $\tau = 0.7$. Tasks scoring below the threshold are rejected and sent back for replanning.

4.5 Error Analysis

Figure 4 compares the error category distribution between the baseline and VPER.

VPER reduces errors across most categories:

- **Column name errors** drop from 7 to 3, as the Planner’s schema context guides the Executor toward correct column references.
- **Filter condition errors** drop from 8 to 4, as the Replanner can detect and correct failed queries.
- **Cross-source integration errors** drop from 5 to 3, as the Workspace schema map provides a unified view of all data sources.
- **Unstructured data parsing** remains at 5, reflecting the fundamental challenge of extracting structured information from narrative Markdown documents—a limitation that requires dedicated document understanding capabilities beyond VPER’s current scope.

4.6 Case Study: Validator-Triggered Correction

We present a representative case where the Validator successfully identifies and corrects an error.

Task 145 (Medium): “Among the events attended by more than 10 members of the Student Club, how many of them are meetings?”

1. **Planning:** The Planner generates a 3-step plan: (1) read attendance CSV, (2) query event database schema, (3) join and aggregate with Python.
2. **Execution:** The Executor successfully joins attendance and event data, counts attendees per event, filters for > 10 , and identifies 4 meetings.
3. **First Validation:** The Validator scores the answer at 0.65 (below threshold 0.7), noting that the trajectory shows redundant schema inspection that could be consolidated.

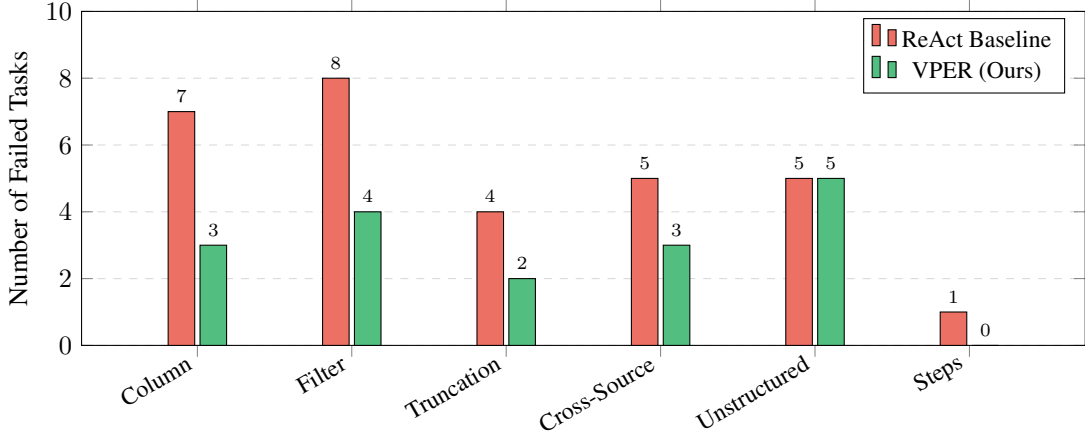


Figure 4: Error category distribution comparison. VPER reduces errors across most categories, with the largest reductions in column name matching, filter condition errors, and cross-source integration failures. Unstructured document parsing remains an unsolved challenge.

4. **Replanning:** The Replanner adjusts the plan to consolidate schema inspection into the analysis step.
5. **Re-execution:** The Executor produces a cleaner result with the correct answer: 4.
6. **Second Validation:** The Validator scores 0.82 (above threshold), and the answer is accepted.

This case demonstrates VPER’s ability to not only catch incorrect answers but also improve answer quality through iterative validation-replanning cycles.

5 Conclusion

We presented VPER, a multi-agent framework that augments the Plan-Execute-Replan paradigm with LLM-as-a-Judge answer verification for reliable data analysis. Through systematic error analysis of the ReAct baseline, we identified three dominant failure modes—myopic execution, lack of replanning, and absence of verification—and addressed each with dedicated modules: a Planner for structured task decomposition, a Replanner for dynamic plan adjustment, and a Validator for pre-submission answer verification.

Experiments on a 50-task benchmark demonstrate that VPER achieves 44.0% accuracy (69.2% relative improvement over baseline) while reducing the failure rate from 6.0% to 2.0%. The Validator module alone corrects 10.0% of incorrectly submitted answers, validating the effectiveness of answer verification before submission.

Limitations and Future Work. VPER’s current limitations include: (1) difficulty with unstructured document parsing (all Extreme tasks and many Hard tasks involve narrative Markdown documents); (2) the Validator introduces additional latency and API costs due to multiple LLM evaluation calls; (3) the 8.0% false positive rate of the Validator suggests room for improvement in evaluation precision. Future work will explore: integrating dedicated document understanding modules, optimizing Validator efficiency through distilled evaluator models, and extending the framework to support multi-turn interactive data analysis.

References

- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022a.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023.
- Shunyu Yao, Dian Li, Jeffrey Zhao, et al. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36, 2023.
- Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, et al. A survey on llm-as-a-judge. *The Innovation*, 2024.
- Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757, 2022b.
- Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.
- Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Cote, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. Alf-world: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 2609–2634, 2023.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- Yohei Nakajima. Babyagi. <https://github.com/yoheinakajima/babyagi>, 2023.
- Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37:126544–126565, 2024.
- Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 3102–3116, 2023.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *The Twelfth International Conference on Learning Representations*.
- Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, Boxi Cao, and Le Sun. Toolalpaca: Generalized tool learning for language models with 3000 simulated cases. *arXiv preprint arXiv:2306.05301*, 2023.